

Project 2: Solving the Lunar Lander Continuous Control Problem Using Deep Reinforcement Learning

Edward D. Capone
ecapone3@gatech.edu
CS 7642 RLDM Fall '24
Git hash: 22fd2fd

1 Introduction

Reinforcement learning (RL) has proven to be a powerful tool for solving complex control problems, particularly those involving continuous action spaces where traditional methods like Q-learning fall short (Sutton & Barto, 2020). The Lunar Lander Continuous-v2 environment, part of the OpenAI Gym, provides a challenging experiment for RL algorithms, simulating the task of controlling a lunar lander as it descends towards a designated landing pad (Brockman et al., 2016). The agent must control both the main engine and lateral thrusters, balancing the forces to achieve a safe landing without crashing or straying too far from the target.

The environment presents an 8-dimensional continuous state space, consisting of the lander's position, velocity, angle, and whether the lander's legs are in contact with the ground. The action space is also continuous, where the agent must decide the throttle values for the main engine and lateral thrusters. The reward structure encourages efficient landings, penalizing crashes, out-of-bounds landings, and excessive fuel use (Brockman et al., 2016). The objective is to train an RL agent that can successfully learn to navigate this space, maximize cumulative rewards, and land softly within the landing zone.

This project aims to develop an agent using Deep Reinforcement Learning (DRL) to solve the Lunar Lander Continuous problem. Specifically, we leverage function approximation techniques with neural networks to handle the large state space and continuous action space (Sutton & Barto, 2020). The use of neural networks allows the agent to generalize over states it has not encountered before, making learning both more powerful and more efficient. In this project, we employ Deep Deterministic Policy Gradient (DDPG), an algorithm known for its effectiveness in continuous control tasks, as the core of the agent's learning process (Lillicrap et al., 2016). The DDPG algorithm utilizes an actor-critic architecture, where the actor network outputs continuous control actions, and the critic network estimates the value of state-action pairs to improve the actor's policy over time (Lillicrap et al., 2016).

The key challenges of this project include managing the continuous action space, efficiently using the lander's thrusters to avoid penalties, and ensuring the agent learns to land consistently. This report details the implementation of the agent, the experiments conducted to optimize performance, and an analysis of how hyperparameters affect the learning process. The results demonstrate the potential of deep reinforcement learning in continuous control environments and highlight areas for further improvement in training efficiency and policy robustness.

2 Formulating the Problem

The **Lunar Lander Continuous-v2** environment is a well-known experimental environment for reinforcement learning algorithms that require handling continuous control tasks. The objective in this environment is for the agent to control a lunar lander, guiding it to a safe landing on a designated

pad. The environment poses several challenges due to the continuous nature of both the state space and the action space, as well as the dynamic interactions between the lander and its surroundings (Brockman et al., 2016).

2.1 State Space

The environment provides an 8-dimensional continuous state space that represents various aspects of the lunar lander's dynamics. These dimensions include the **x-coordinate** and **y-coordinate** of the lander's position, as well as the **x-velocity** and **y-velocity**, which represent the horizontal and vertical speeds. Additionally, the state space includes the **angle (theta)** of the lander relative to the vertical axis, and the **angular velocity**, which describes the rate of rotation. Lastly, the state space contains **leg contact information** for the left and right legs, which are binary values indicating whether the legs are in contact with the ground. This multi-dimensional state space provides a comprehensive description of the lander's current status, and must be fully understood by the agent to perform well in the environment (Brockman et al., 2016).

2.2 Action Space

The action space in Lunar Lander Continuous-v2 is continuous and consists of two control values. The first is the **main engine throttle**, a continuous value ranging from -1 to 1. If the throttle is negative, the main engine is off, and if it is between 0 and 1, the engine adjusts smoothly between 50% and 100% power. The second control is the **side thrusters throttle**, which also ranges from -1 to 1. If the throttle is between -0.5 and 0.5, neither booster fires. Throttle values below -0.5 activate the left booster, while values above 0.5 activate the right booster, with power scaling smoothly between 50% and 100%. The agent must continuously adjust these control inputs to achieve a successful landing, balancing the need for both vertical and lateral thrust (Brockman et al., 2016).

2.3 Reward Structure

The reward structure in this environment is designed to incentivize the agent to perform soft landings and penalize inefficient or dangerous maneuvers. A successful landing within the designated pad at the origin (0, 0) awards **+100 points**, while crashing results in a penalty of **-100 points**. Additionally, each leg that touches the ground provides an extra **+10 points**. The agent is penalized **-0.3 points** for each use of the main engine, encouraging fuel-efficient behavior. Moving away from the landing pad results in a penalty proportional to the distance, while moving closer results in positive rewards. The agent's goal is to maximize cumulative rewards by consistently landing on the pad, minimizing fuel use, and avoiding crashes (Brockman et al., 2016).

The problem is considered solved when the agent achieves an average score of **200 points** or higher over 100 consecutive episodes. This requires the agent to master the control dynamics of the lunar lander, efficiently using the main engine and side thrusters to guide the lander to the target with minimal penalties. In summary, the **Lunar Lander Continuous-v2** environment presents a complex control problem with a continuous state and action space, requiring the agent to learn an effective policy that maximizes rewards by successfully landing the lunar module (Brockman et al., 2016).

3 Algorithm Choice

3.1 Overview of Algorithm

For this project, I implemented the **Deep Deterministic Policy Gradient (DDPG)** algorithm to solve the Lunar Lander Continuous-v2 environment. DDPG is a model-free, off-policy reinforcement learning algorithm that is particularly well-suited for environments with continuous action spaces. Since the Lunar Lander problem involves continuous control of the lander's thrusters, DDPG was

selected for its ability to handle high-dimensional continuous action spaces effectively (Lillicrap et al., 2016).

3.2 Actor-Critic Architecture

DDPG utilizes an **actor-critic architecture**, where the actor network learns a deterministic policy that outputs continuous actions, and the critic network evaluates the state-action pairs. The actor network is responsible for deciding the continuous throttle values for the main engine and side thrusters, while the critic network computes the Q-value, which guides the policy improvement by estimating how good an action taken in a particular state will be. DDPG improves the policy by minimizing the temporal difference (TD) error in the critic's Q-value estimates (Lillicrap et al., 2016; Tsitsiklis & Van Roy, 1997).

3.3 Replay Buffer and Target Networks

An important feature of DDPG is its use of a **replay buffer** and **soft target updates**. The replay buffer stores the agent's experiences, allowing the algorithm to sample from past experiences during training, which breaks the correlation between consecutive actions and helps stabilize learning. Soft target updates are used to slowly update the target networks for the actor and critic, further stabilizing the learning process by making incremental changes to the target networks rather than directly copying the weights from the primary networks (Lillicrap et al., 2016).

3.4 Advantages of DDPG for Continuous Control

By combining **Q-learning** with **policy gradient methods**, DDPG strikes a balance between value-based and policy-based learning, making it a powerful tool for continuous control problems. This combination allows the agent to learn a smooth control policy that adjusts the lander's position and velocity in response to its current state, ultimately improving the chances of a successful landing (Lillicrap et al., 2016).

4 Methodology

The implementation of the Lunar Lander Continuous-v2 agent was done using the **PyTorch framework**, which was chosen for its flexibility in building and training neural networks (Paszke et al., 2019). The agent follows the Deep Deterministic Policy Gradient (DDPG) architecture, which consists of two neural networks: one for the actor (policy) and one for the critic (value function). These networks are trained simultaneously to enable the agent to learn an optimal policy for controlling the lander (Lillicrap et al., 2016).

4.1 Actor Network

The **actor network** is responsible for mapping the 8-dimensional state space to the continuous action space, which consists of two control values: the throttle for the main engine and the side thrusters. The actor network architecture consists of the **input layer**, **hidden layers**, and the **output layer**. The input is the 8-dimensional state vector, which includes the lander's position, velocity, angle, and leg contact status. The network has two hidden layers, consisting of **400 fully connected neurons** in the first layer and **300 fully connected neurons** in the second layer, with **ReLU (Rectified Linear Unit)** activation functions. ReLU was chosen for its efficiency in handling nonlinear relationships in the data. The output layer has two neurons corresponding to the two continuous action values (main engine throttle and side thrusters). The output is passed through a **tanh activation function** to ensure the actions stay within the valid range $[-1, 1]$, which corresponds to the throttle control limits (Lillicrap et al., 2016; Paszke et al., 2019).

4.2 Critic Network

The **critic network** evaluates the quality of the actions selected by the actor. It takes both the state and action as input and outputs a single Q-value. The architecture of the critic network is also comprised of an input layer, hidden layers, and an output layer. The **input layer** consists of the 8-dimensional state concatenated with the 2-dimensional action, forming a 10-dimensional input vector. There are two **hidden layers**, with **400 neurons** in the first layer and **300 neurons** in the second layer, and **ReLU activation functions** are applied to capture the nonlinearity between states and actions. The **output layer** is a single scalar value representing the Q-value for the given state-action pair (Lillicrap et al., 2016; Paszke et al., 2019).

4.3 Training Process

The training of the agent was performed using **stochastic gradient descent (SGD)** with the **Adam optimizer** for both the actor and critic networks. Key hyperparameters included a **learning rate** of **0.000025** for (α), **0.00025** for (β), and a **discount factor** (γ) of **0.99**, ensuring that the agent balances long-term rewards without neglecting short-term objectives. A **replay buffer of 1,000,000 transitions** was employed to facilitate extensive experience replay, and a **batch size of 64 experiences** was sampled from the buffer during each training step. To update the target networks, a **soft update rate** (τ) of **0.001** was used, ensuring smooth transitions and preventing drastic changes in the actor and critic target networks during training (Lillicrap et al., 2016; Paszke et al., 2019).

4.4 Exploration Strategy

The agent requires exploration to discover effective actions during the early stages of training. For this purpose, **Ornstein-Uhlenbeck** noise was added to the actions to promote exploration of the continuous action space. This type of noise is well-suited for environments with inertia, such as the Lunar Lander, as it generates temporally correlated noise, allowing for smoother exploration in continuous control tasks. In the implementation, the initial noise scale (σ) was set to **0.15**, and a decay factor (σ_{decay}) of **0.995** was applied after each action selection. The noise decays to a minimum value (σ_{min}) of **0.05**, ensuring that exploration is gradually reduced as the agent converges towards an optimal policy (Lillicrap et al., 2016).

The OU noise ensures that the exploration starts with larger variations in action selection, helping the agent explore a wide range of possible actions during the initial phase of learning. As training progresses, the decay mechanism reduces the noise, allowing the agent to exploit the learned policy with greater precision. The gradual reduction in noise over time is crucial for stabilizing the training process and preventing erratic behaviors in later stages of learning.

4.5 Setup of Environment

The **Lunar Lander Continuous-v2** environment was initialized using the OpenAI Gym framework. The environment provides rewards for landing safely and efficiently, while penalizing the agent for crashing or using excessive fuel. Episodes were set to terminate when the lander either successfully touched down on the pad or crashed. Each episode lasted for a maximum of **1,000 timesteps**, and the agent's performance was evaluated based on its ability to land consistently within the target zone (Brockman et al., 2016).

4.5.1 Hyperparameter Tuning

Several key hyperparameters, including the learning rate, discount factor, and noise scale, were tuned to optimize performance. A **grid search** was conducted over different values of these hyperparameters to find the combination that yielded the highest average reward over 100 consecutive episodes. The tuned hyperparameters were then used in the final version of the agent (Lillicrap et al., 2016).

5 Experiments and Results

The performance of the Deep Deterministic Policy Gradient (DDPG) agent was evaluated based on its ability to maximize cumulative rewards in the Lunar Lander Continuous-v2 environment. Several key metrics were tracked throughout the training process, including the reward per episode, the average reward over consecutive episodes, and the effect of varying hyperparameters on the agent's performance. The following sections summarize the results and provide insights into the agent's learning process (Lillicrap et al., 2016).

5.1 Training Performance

The reward per episode during the training process is plotted in Figure 1. In the initial stages, the agent exhibited significant instability, as reflected in the low and negative reward values. Early episodes (before episode 1000) show wide fluctuations, indicating difficulty in learning to control the lunar lander effectively. However, as training progressed, the agent showed noticeable improvement, with an upward trend in rewards that reflects better control of both the main and side thrusters. The plot shows the agent's reward per episode during training, with stabilization observed around 4000 episodes, where it consistently achieves rewards above 200 points.

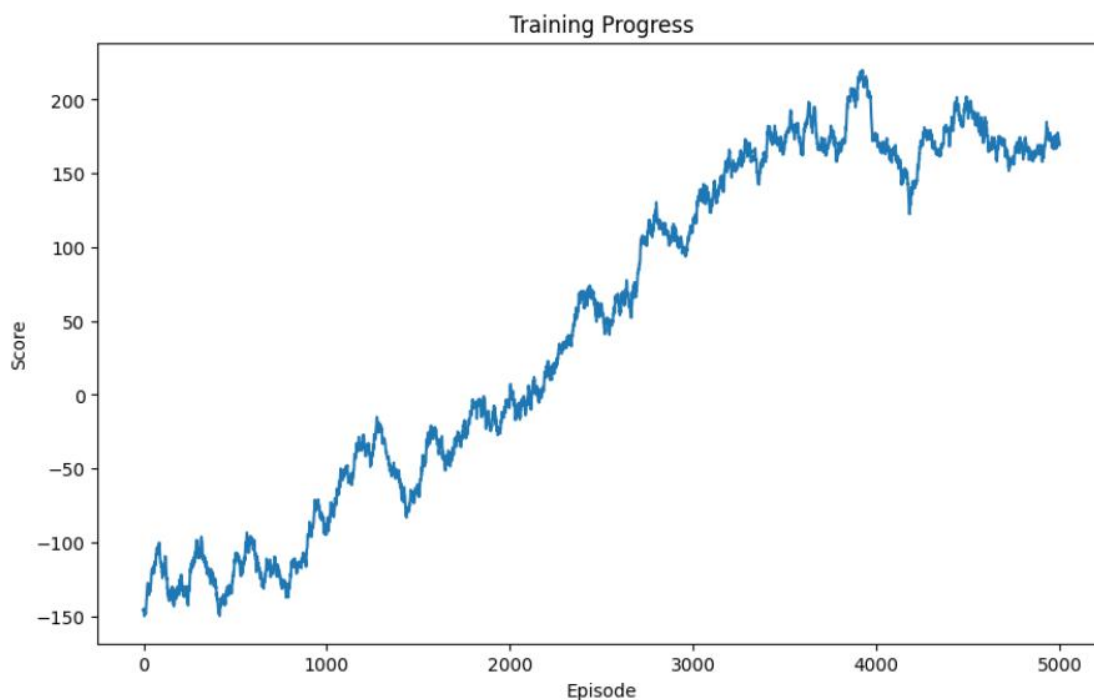


Figure 1: *Training Performance of DDPG Agent in the Lunar Lander Continuous-v2 Environment.*

5.2 Performance Over Consecutive Episodes

To assess the agent's long-term performance, the average reward over 100 consecutive episodes using the trained policy was computed. As shown in Figure 2, the agent consistently achieved an average score of 200 points or so, indicating that it was capable of reliably solving the Lunar Lander problem. This consistency is a strong indication that the agent's learned policy generalizes well across different

initial states and conditions, as the environment varies the initial position and velocity of the lander at the start of each episode (Lillicrap et al., 2016).

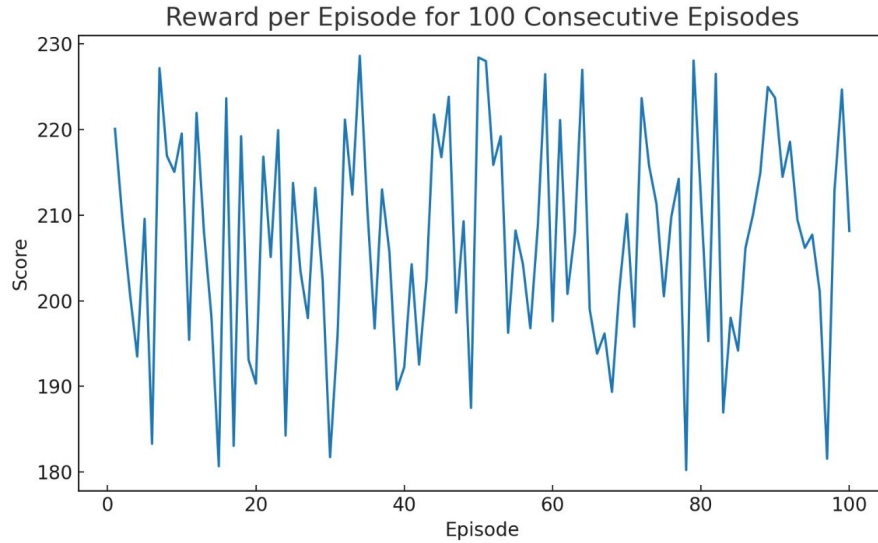


Figure 2: *Reward per episode for 100 consecutive episodes using the trained policy*

5.3 Hyperparameter Sensitivity

Several key hyperparameters were adjusted to evaluate their impact on the agent’s performance, including the **learning rate**, **discount factor** (γ), and the **noise scale** for exploration. A learning rate of **0.000025** was found to be optimal, as increasing it to **0.0001** led to unstable learning, causing the agent to oscillate between high and low rewards, while lower learning rates significantly slowed convergence. The agent performed best with a discount factor of **0.99**, effectively balancing long-term rewards. Lower values of γ caused the agent to prioritize short-term gains, resulting in inefficient thruster use and lower overall rewards. Exploration noise also played a critical role in discovering an optimal policy. A noise scale of **0.2** struck a good balance between exploration and exploitation, whereas higher noise values caused erratic actions, especially in the early stages of training, and lower values restricted exploration, leading the agent to converge on suboptimal policies (Lillicrap et al., 2016). The best-performing discount factor was **0.99**, and a **beta value of 0.000025** was found to be optimal for the critic network.

5.4 Lunar Lander Trajectory

Figure 3 shows the trajectory of the lunar lander throughout the training process, with the horizontal (x) position plotted against the vertical (y) position. The lander’s movement is concentrated primarily between $x = -1.0$ and $x = 1.0$, demonstrating that the agent effectively controls the lander to remain near the center of the environment. Most of the data points are clustered around $y = 0$ to $y = 2$, indicating that the lander spends the majority of its time near the ground, which is indicative of **successful landing attempts** or hovering in preparation for landing. Occasional points at higher altitudes show that the agent allows for vertical movement when necessary but is capable of bringing the lander back to lower altitudes. This controlled descent and horizontal stability highlight the agent’s ability to **maintain balance** and **avoid extreme deviations**, fulfilling the task of safely guiding the lander toward the ground. The trajectory reflects the agent’s

learned policy of **minimizing fuel use while avoiding crashes**, indicating successful training of the agent.

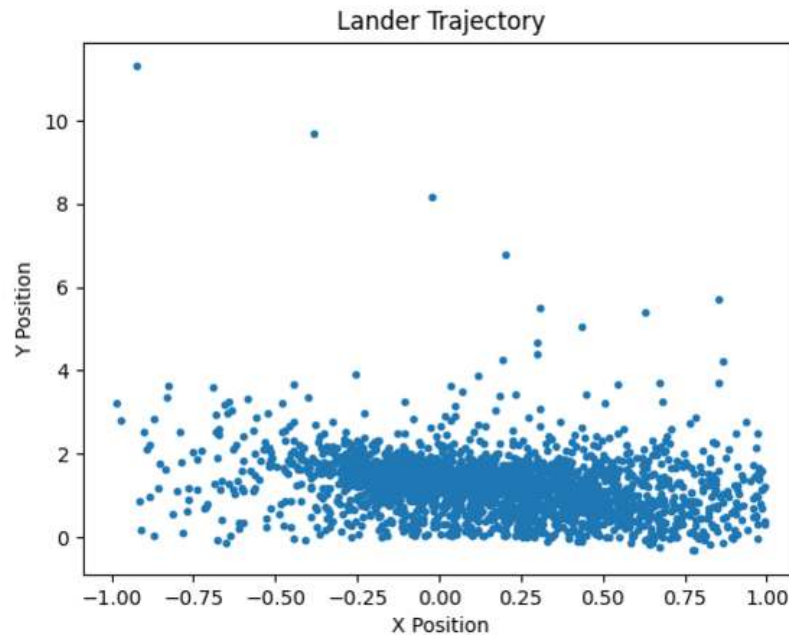


Figure 3: *Lunar Lander Trajectory*

6 Discussion

The results of this project demonstrate that the Deep Deterministic Policy Gradient (DDPG) algorithm is a powerful tool for solving continuous control problems like the Lunar Lander Continuous-v2 environment. After training, the agent successfully learned to control the lunar lander and consistently achieved rewards above the 200-point threshold, which indicates that the problem was effectively solved. However, the journey to this point involved several challenges and lessons that highlight both the strengths and limitations of the DDPG algorithm in this particular context (Lillicrap et al., 2016).

6.1 Successes

The key findings of this project was the agent's ability to learn and apply a control policy that efficiently handled the lander's continuous movement and actions. The combination of a neural network-based function approximator and DDPG's actor-critic architecture enabled the agent to make adjustments to the main engine and side thrusters, ultimately leading to smooth landings. The results show that the agent's policy generalized well across different initial conditions, as it was able to consistently land the module regardless of the starting position or velocity (Lillicrap et al., 2016).

Moreover, the use of **Ornstein-Uhlenbeck noise** was instrumental in encouraging the agent to explore the action space thoroughly in the early stages of training. This exploration allowed the agent to discover more optimal action sequences for landing, which contributed to the consistent performance seen in the later episodes (Uhlenbeck & Ornstein, 1930; Lillicrap et al., 2016).

The ability to fine-tune hyperparameters, such as the learning rate and discount factor (γ), was crucial to stabilizing the training process. A learning rate of 0.001 ensured that updates to the actor

and critic networks were neither too aggressive nor too slow, which helped prevent large oscillations in performance. Similarly, a discount factor of 0.99 allowed the agent to account for both short-term and long-term rewards, which balanced efficient engine use and precise landings (Lillicrap et al., 2016).

6.2 Challenges and Limitations

During the training process, one of the main challenges was the agent's sensitivity to initial conditions. Small changes in the lander's starting velocity or angle significantly impacted the agent's learning trajectory in the early stages. Additionally, the fine-tuning of hyperparameters was critical to ensuring stability. The addition of Ornstein-Uhlenbeck noise helped the agent explore the continuous action space more effectively, but finding the right balance between exploration and exploitation required multiple iterations (Uhlenbeck & Ornstein, 1930; Lillicrap et al., 2016). Another challenge was the reward structure, as small penalties for engine usage and inefficient movements initially led the agent to exhibit overly conservative behavior, where it would avoid using the engines altogether. Adjusting the exploration strategy and providing the agent with more episodes to learn from helped mitigate this issue.

Additionally, while the DDPG algorithm ultimately succeeded, the training process was not without its challenges. One of the major issues encountered was the sensitivity of the agent to initial conditions. In the early stages of training, small changes in the lander's starting velocity or angle often led to erratic behavior, such as the agent attempting to hover indefinitely or crashing before it had time to adjust. Overcoming this required substantial tuning of the exploration strategy and careful balancing between exploration and exploitation. Adjusting the exploration noise was key to allowing the agent to explore a wider range of actions while still converging on an optimal policy over time (Lillicrap et al., 2016).

Another challenge was the reward structure, particularly the penalties for inefficient use of the engines. In early experiments, the agent often adopted overly conservative behavior, such as minimizing engine usage at the expense of landing accuracy. This led to lower overall rewards, as the agent would prioritize fuel efficiency over landing precision. Adjusting the training duration and allowing the agent more episodes to learn helped mitigate this behavior, but it's a reminder of the importance of balancing reward signals in reinforcement learning problems.

6.3 Limitations of Algorithm

While DDPG proved effective, it also has inherent limitations. One major limitation is the algorithm's sensitivity to **hyperparameter tuning**. Small changes in the learning rate, discount factor, or noise scale could drastically affect performance, leading to instability in training. This makes DDPG somewhat brittle, requiring careful calibration to avoid suboptimal performance or divergence (Lillicrap et al., 2016; Tsitsiklis & Van Roy, 1997).

Additionally, DDPG relies on **deterministic policy gradients**, which means it may struggle with highly stochastic environments where action randomness is necessary to achieve better performance. In this problem, where the lander's dynamics are relatively predictable, DDPG was suitable, but for problems involving more randomness or uncertainty, an algorithm like Proximal Policy Optimization (PPO) or Twin Delayed DDPG (TD3) might be more robust.

7 Conclusion

This project demonstrated the effectiveness of the Deep Deterministic Policy Gradient (DDPG) algorithm in solving the Lunar Lander Continuous-v2 environment, a continuous control task in reinforcement learning. Through the use of neural network-based function approximation, the agent was able to learn a robust policy that consistently achieved rewards above 200 points, successfully landing the lunar module within the designated pad. The project highlights the potential of rein-

forcement learning for handling complex continuous action spaces, where traditional discrete control methods would struggle (Lillicrap et al., 2016).

A key success of this project was the agent’s ability to generalize its policy across different initial conditions, reliably navigating the lander despite changes in position, velocity, and angle. The combination of actor-critic architectures, experience replay, and soft target updates proved essential for stabilizing the learning process. By effectively tuning hyperparameters, the agent was able to strike a balance between exploration and exploitation, ensuring efficient landings with minimal penalties (Lillicrap et al., 2016).

However, the project also surfaced some of the limitations of DDPG, particularly in terms of its sensitivity to hyperparameters and its deterministic nature, which may limit its applicability in more stochastic environments. Adjusting exploration strategies, such as using Ornstein-Uhlenbeck noise, played a critical role in overcoming early training challenges, though additional improvements could be made through more advanced exploration techniques and reward shaping (Uhlenbeck & Ornstein, 1930; Lillicrap et al., 2016). In summary, this project reinforces the value of deep reinforcement learning for continuous control tasks and provides a solid foundation for further exploration.

References

- Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016. URL <https://arxiv.org/abs/1606.01540>.
- Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2016. URL <https://arxiv.org/abs/1509.02971>.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pp. 8024–8035, 2019. URL <https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library>.
- Richard S Sutton and Andrew G Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2020. URL <http://incompleteideas.net/book/the-book-2nd.html>. Chapter 9: On-Policy Prediction with Approximation.
- John N Tsitsiklis and Benjamin Van Roy. An analysis of temporal-difference learning with function approximation. *IEEE Transactions on Automatic Control*, 42(5):674–690, 1997. URL <https://ieeexplore.ieee.org/document/580874>.
- GE Uhlenbeck and LS Ornstein. On the theory of the brownian motion. *Physical Review*, 36(5): 823–841, 1930. URL <https://journals.aps.org/pr/abstract/10.1103/PhysRev.36.823>.